

Deploy Java App to Minikube Automated with Jenkins

1. Overview

Automating the deployment of a Java application to Minikube using Jenkins involves building the application, creating a Docker image, pushing it to a container registry, and deploying it to Minikube using Kubernetes manifests.

2. Key Concepts

A. Jenkins Pipeline

Jenkins automates the CI/CD process using a declarative pipeline. The pipeline consists of multiple stages such as:

- **SCM Checkout:** Fetches code from a repository (GitHub/GitLab).
 - **Build & Test:** Uses Maven (mvn package) to compile and test the Java application.
 - **Docker Build & Push:** Builds a Docker image of the application and pushes it to Docker Hub.
 - **Deploy to Minikube:** Uses kubectl to apply Kubernetes deployment and service files.
-

B. Minikube

Minikube is a lightweight Kubernetes cluster for local development and testing. It allows developers to run Kubernetes locally and deploy applications without needing a cloud-based cluster.

Commands to Start Minikube:

```
sh
```

```
CopyEdit
```

```
minikube start
```

```
kubectl cluster-info
```

```
kubectl get nodes
```

C. Docker

Docker is used to package the Java application into a container image, making it portable and easy to deploy across environments.

Dockerfile Example:

```
dockerfile
```

```
CopyEdit
```

```
FROM openjdk:11
```

```
COPY target/webapp.jar /app/webapp.jar
```

```
WORKDIR /app
```

```
CMD ["java", "-jar", "webapp.jar"]
```

D. Kubernetes Deployment

Kubernetes YAML files define how the application should be deployed inside the Minikube cluster.

Deployment YAML Example:

```
yaml
```

```
CopyEdit
```

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
```

```
  name: webapp
```

```
spec:
```

```
  replicas: 1
```

```
  selector:
```

matchLabels:

app: webapp

template:

metadata:

labels:

app: webapp

spec:

containers:

- name: webapp

image: nidharsan8008/war-jenkins1

ports:

- containerPort: 8080

Apply Deployment:

sh

CopyEdit

kubectl apply -f deployment.yml

kubectl get pods

pipeline {

agent any

stages {

stage('Clone Git Repository') {

steps {

git 'https://github.com/NIDHARSAN-V/Devops_Guvi_WAR.git'

```
    }  
}
```

```
stage('Build WAR File') {  
    steps {  
        sh 'mvn clean package' // Build WAR file using Maven  
    }  
}
```

```
stage('Build Docker Image') {  
    steps {  
        script {  
            sh 'docker build -t warimage-jenkins .'  
            sh 'docker tag warimage-jenkins nidharsan8008/warimage-jenkins1'  
        }  
    }  
}
```

```
// stage('Run Docker Container') { // Optional stage to test the container  
//     steps {  
//         script {  
//             sh 'docker run -d -p 8888:8080 --name war-container-jenkins warimage-jenkins'  
//         }  
//     }  
// }  
// }
```

```
stage('Push to Docker Hub') {  
    steps {  
        script {  
            withDockerRegistry(credentialsId: 'dockerhub_token', url:  
'https://index.docker.io/v1/') {  
                sh 'docker push nidharsan8008/warimage-jenkins1'  
            }  
        }  
    }  
}
```

```
stage('Deploy with deployment.yml') {  
    steps {  
        withKubeConfig(caCertificate: '', clusterName: 'minikube ', contextName: 'minikube',  
credentialsId: 'MiniKube_ID', namespace: '', restrictKubeConfigAccess: false, serverUrl:  
'https://192.168.49.2:8443') {  
            sh 'kubectl apply -f deployment.yml'  
        }  
    }  
}
```

```
// stage('Test') {
```

```
//     steps {
```

```
//      withKubeConfig(caCertificate: "", clusterName: 'minikube ', contextName: 'minikube',
credentialsId: 'MiniKube_ID', namespace: "", restrictKubeConfigAccess: false, serverUrl:
'https://192.168.49.2:32274') {
```

```
    //      sh "
```

```
    //    }
```

```
    // }
```

```
    //}
```

```
}
```

```
}
```

